

A sequence machine built with an asynchronous spiking neural network

J. Bose*, S.B. Furber[†], J.L. Shapiro[†]

School of Computer Science

University of Manchester, Manchester M13 9PL, UK

*E-mail: joy.bose@cs.manchester.ac.uk

[†]E-mail: [steve.furber, jon.shapiro]@manchester.ac.uk

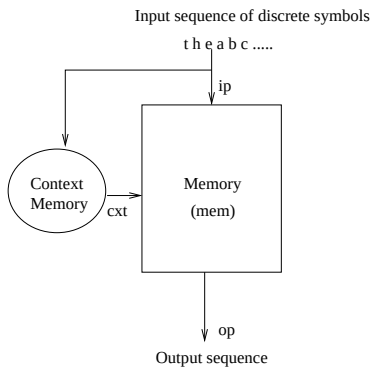


Fig. 1. Basic design of a sequence memory

Abstract—In this paper we present the design of a sequence machine, a finite state automaton that can learn and predict sequences of symbols, built out of a network of asynchronous spiking neurons. We concentrate on aspects of building synchronous systems out of asynchronous neural components, such as stability and coherence of spike bursts forming symbols.

I. INTRODUCTION

In this paper we design a synchronous high level system, a sequence machine, out of low level asynchronous components, spiking neurons. A sequence machine is a finite state automaton that can store and recall temporal sequences of symbols. We are interested in the design of an on-line sequence machine that can learn best from a single presentation of a sequence and which can also do prediction of the next symbol in the sequence. Each time the machine sees a new input symbol, it makes a prediction of the next input symbol in the sequence. If it encounters a familiar sequence of symbols, it can ‘lock on’ the sequence and the prediction will be correct, else it will learn the association of the symbols and the next time it sees the sequence the prediction will be improved.

We will build the sequence machine using an associative memory, a type of neural network which can store associations and can learn in a single pass. Our machine also needs to have a memory for storing the history or context of the sequence, apart from the memory for storing the associations. The structure of a basic sequence machine is shown in figure 1. The machine is input a continuous chain of symbols, and its task is to learn the subsequences in this chain, and predict

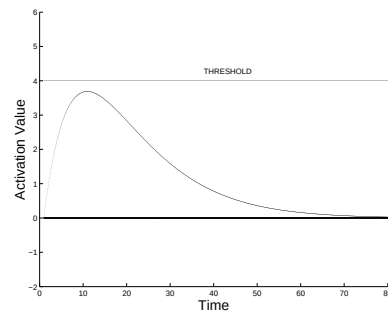


Fig. 2. Plot of a typical neural activation with time, when the neuron receives a single input spike at $t=0$

the next symbol for each input symbol.

The main reason why we are interested in building this system out of spiking neurons is to study the dynamics of engineering high level systems out of spiking neurons. Also, memories built out of neural networks are error tolerant, since the memory is distributed over many locations.

In building such a system, there are certain issues we need to consider: how do we represent the temporal information of the history of the sequence spatially, to store them in the neural network? How do we synchronise the spike bursts of the neural systems to make sense of them as symbols to be used in the sequence machine? In this paper, we analyse these issues and suggest solutions.

II. SPIKING NEURONS

Spiking neurons [2] are a type of artificial neural networks, where all information is transmitted by means of stereotyped spikes emitted by neurons. Since the spikes are similar in shape, the only meaningful information is in their timing of spiking. Each neuron acts as an integrator, integrating spikes from input neurons and emitting output spikes if input spikes cause the activation of the neuron to exceed a threshold. Once a neuron has fired a spike, its activation is reset to a negative value and decays back to the resting value in the absence of more input spikes. The fired spike is propagated to neurons connected to it and the process continues.

A. Model of spiking neuron

We have chosen a rate driven leaky integrate and fire (RDLIF) model of spiking neuron, which is similar to the

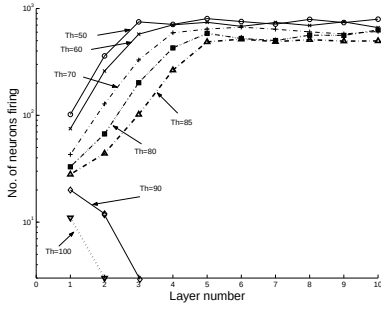


Fig. 3. Plot of the variation of the number of neurons firing in each layer with the threshold. The system behaviour switches from spiking activity increasing with each successive layer for thresholds ≤ 85 to dying out for thresholds > 85 .

standard leaky integrate and fire (LIF) model [2], with the difference that input spikes to a neuron increase the rate, or first derivative, of the activation of the neuron, rather than the activation itself, as in the standard LIF model. When the activation reaches a threshold, the neuron is said to have fired a spike and both the activation and rate are reset. In the absence of incoming spikes, both the activation and its rate decay to resting values, the rate of decay being governed by their respective time constants. Figure 2 shows the response kernel of a single RDLIF neuron that has an input spike at time $t=0$. The activation initially rises due to the rate increase caused by the input spike, but after a while the decay becomes dominant. If the neuron gets a number of closely spaced input spikes, the activation has a greater chance of crossing the threshold than if the inputs are spaced out. Our RDLIF model is more flexible than the standard LIF model, especially when feed-back loops exist in the network.

III. ENCODING SCHEME AND ITS PROBLEMS

The sequence machine we are trying to build with spiking neurons is a synchronous system dealing with sequences of symbols, which are encoded as spike bursts emitted by layers of spiking neurons firing asynchronously. There are two levels of sequence we have used here: the higher level sequence of symbols, and the lower level sequence of spikes forming symbols. We need to first define a symbol in terms of the fired spikes. Here we define a symbol to be encoded as a specific temporal order of firing of spikes emitted by a layer of neurons. This encoding using the temporal order of firing is called rank order coding and was first used by Thorpe [3]. For example, a symbol A might be represented as a firing order of 3 neurons in a layer, such as 2, 1, 3. The total number of neurons in the layer might be more than the number of fired neurons, in what is called N-of-M encoding, a form of self error correcting codes, in which the choice of N firing neurons in a firing burst, out of a total of M neurons in the layer is used to encode the symbol [1]. We have used this ordered N-of-M encoding in our system, as a combination of N-of-M encoding and rank order codes.

However, in using this encoding scheme in our spiking neural system, we need to ensure that the stability, coherence

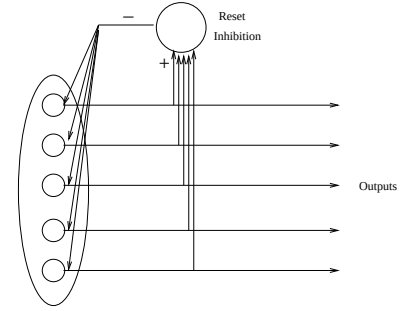


Fig. 4. Use of feed-back reset inhibition to control spiking activity when a certain number of output spikes have fired. This can help in sustaining a desired average level of activity across a population of neurons

and order of the code are maintained when the symbols are propagated across layers of neurons. Stability of the code means that the spike burst should not die out or saturate to an unacceptably high level when it passes through different layers of neurons. Coherence means that different spike bursts forming symbols should not impinge on each other, destroying any information that is being propagated. We need to ensure that there is proper temporal separation maintained between different spike bursts. Finally, we need to make sure that the system is sufficiently tolerant to any disturbance in the order of spike firings, because that is what determines the code for symbols.

A. Stability

We simulated a feed-forward network of layers of spiking RDLIF neurons which had the same average connectivity but different weights in each layer. We found that as we increased the threshold for different runs of the experiment, the spiking activity of the successive layers switched abruptly from dying out to saturating to an unacceptably high level, as shown in figure 3. There was no threshold such that a stable activity of a spike burst could be maintained. To solve this problem of stability, we introduced feed-back reset inhibition on the layers, implemented by a neuron that was fed the output spikes from a layer and which cut off the activity once a certain number of spikes had fired in the layer, as shown in figure 4. We found that by choosing a threshold that allowed the output spiking activity for the layers to be greater than input activity and then cutting off the activity by feed-back reset inhibition, it was possible to get stable propagation of spiking activity. This feed-back inhibition can implement N-of-M codes, by firing a resetting spike when N of total M neurons in a layer have fired.

B. Coherence

We found that using feed-back inhibition in our simulated model of layers of RDLIF neurons, the temporal dispersions of the spiking bursts of each layer fluctuated within a range, as shown in figure 5. So we could ensure that the inter-burst separation is large compared to this dispersion of one burst by inserting axonal delays, and so could ensure that the bursts were coherent and symbols did not interfere with each other.

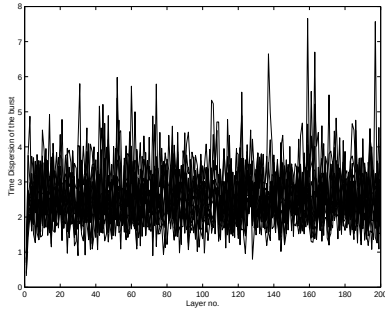


Fig. 5. The time dispersions of the spike bursts across different layers of neurons in the simulated model

C. Interpreting and maintaining the order of spike firings

Since in our model we used rank order coding to encode the symbols as spike bursts, we also need a way to check how accurate the codes are, or how similar is a specific order of firings to a symbol in the machine's alphabet, so that we can identify or decode it, and make the memory tolerant to small errors in order by introducing a suitable threshold on the similarity. We have selected dot product of two vectors as a measure of similarity of their corresponding rank-ordered N-of-M codes. If two vectors are identical, their normalised dot product will be 1, else it would be the cosine of the angle between them. We represent a rank ordered N-of-M code as a vector of M components, with the component corresponding to the neuron firing first in the layer getting a weight of 1, the second a weight of α , where $\alpha < 1$, the third α^2 and so on, in a geometric progression. For example, by taking $\alpha=0.9$, the vector [1 0 0.81 0.9 0] describes a 3-of-5 code in which the neurons numbered 1,4,3 fire in order. By using this measure of abstraction we have a way of judging the similarity of vectors as the normalised dot product of the vectors and decoding a spike burst as a symbol whose corresponding vector is closest to that of the burst.

IV. THE MEMORY USED IN THE SEQUENCE MACHINE

In our model we have used a modified Kanerva sparse distributed memory (SDM) [4] using N-of-M codes [1] with rank order as the encoding, as described in the previous section. SDM's are so called because the storage of the associations is sparse compared to the theoretical maximum as in ordinary content addressable memories, and distributed over a number of neurons rather than localised at one place. N-of-M SDM's have been shown to be scalable and error tolerant memories [1], which was the reason for our selecting them for our memory.

The N-of-M SDM has two layers of neurons: an address decoder layer, whose primary purpose is to cast the input symbol into high dimensional space to make it more linearly separable, and a second was a correlation-matrix layer [5] called the data store, which associates the first symbol as decoded by the first neural layer, with the second symbol. Learning takes place only in this layer, while the weights of the first address decoder layer stay constant. The number of

address decoder neurons is much greater than the number of input neurons. The large size of the address decoder layer is what makes such memories scalable, else they would be identical to correlation matrix memories.

The operation of the memory can be described as follows: Suppose we need to store an association between a symbol A (address) and a symbol B (data), in the write phase. Both are encoded as rank ordered N-of-M coded vectors. The encoded symbol A is passed through the address decoder, which calculates the output code by taking the dot product with the weight matrix and applying a nonlinear ordered N-of-M encoding. The resulting high-dimensional output is fed to the data memory. The data memory writes the association between the high dimensional address decoder output and the encoded symbol B by applying a nonlinear function, in which the new weight matrix is the maximum of the old weight matrix and the outer product of the encoded address decoder output and the encoded input. The connection matrices of data memory and address decoder have real values between 0 and 1. In the read phase, when we need to read out a previously written association, we supply the address A, which is similarly encoded and passed through the address decoder layer, whose output vector is passed through the data memory and the output is calculated by taking the dot product of this vector with the weight matrix and applying a similar nonlinear ordered N-of-M transformation. The symbol closest to the output vector can be determined by talking the normalised dot product of the vector with every symbol in the alphabet and selecting the symbol which is most similar, i.e whose dot product is maximum. This task can also be accomplished by a competitive layer of neurons, with one neuron for each symbol in the alphabet. The similarity of the resulting output vector can be compared with the initial stored data B to measure the accuracy of the retrieval for testing purposes.

V. ENCODING THE PAST HISTORY

In building a sequence machine, we need to find a way to encode the past history of the sequence, since the memory will need to predict the next symbol in the sequence based on the symbols it has seen previously (the history) and the present input symbol. We want our sequence machine to be such that it should be influenced by the past history of the sequence in predicting the successor symbol, if there are two very similar sequences. However, it should also be able to converge on to a familiar sequence in a short while, even if the history is not identical. So we need the present inputs to have a greater weight than past inputs, in determining the next predicted symbol. There are two ways we can encode the history or context of a symbol: by a fixed time window of the past, or by a nonlinear function of the past symbols. The time window or shift register of length N model stores the past N symbols it has seen, and the next prediction is based on the past N symbols as well as the present symbol. However, if there are two sequences such that the number of common symbols is greater than the length of the time window, this model will fail. The other way to encode the history is by a nonlinear function,

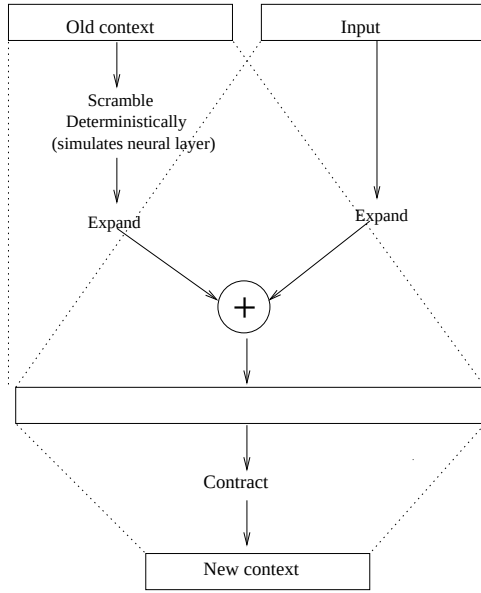


Fig. 6. Formation of the new context from the old context and the input in the combined model. The model has aspects of both the neural layer and the shift register.

which can be implemented by a neural layer representing the context, which is fed as input the previous context and present input, and calculates the output based on a nonlinear encoding of both. The problem with this neural layer or nonlinear function model is that we cannot give a greater weightage to the present input in determining the prediction, which is desirable in actual sequences.

We combine the above two models in a new memory model by using a separate context layer with modulated context, where the new context is determined by both the input and a shifted version of the present context. The new context is formed from the input and old context as follows (see fig. 6): First we scramble of the old context, which is equivalent to passing it through a neural layer. Then the old context is mapped on to a high dimension, expanded and added to the expanded input. The sum is then contracted. The intention is that the result should be strongly dominated by the present input, but should have some bits of the past context in it as well.

VI. TESTS ON THE SEQUENCE MACHINE

We conducted an experiment on the three types of sequence machine (the shift register, neural layer and the combined model) to compare their performance as sequence memories. The results are plotted in figure 8. For each point on the figure we started with a blank memory and input the sequence twice. The memory learns the sequence on the first presentation of the input, and in the second time we compare the predicted output sequence with the input sequence to see how accurate the prediction is. The parameters for the respective models have been optimised. We see that the combined model (solid line) performs the best of the three and obtains near perfect recall.

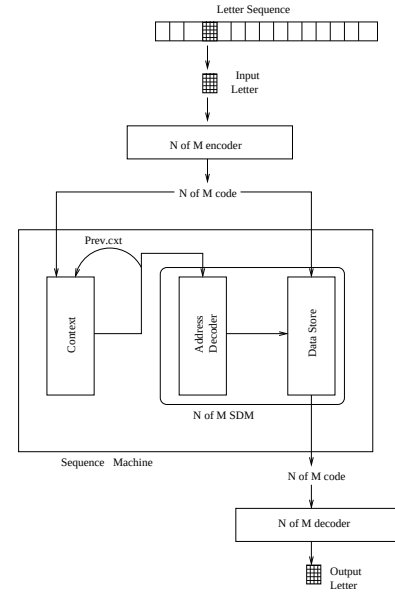


Fig. 7. A sequence machine using an N of M Kanerva type network for scalability, having address decoder, data memory and context layers

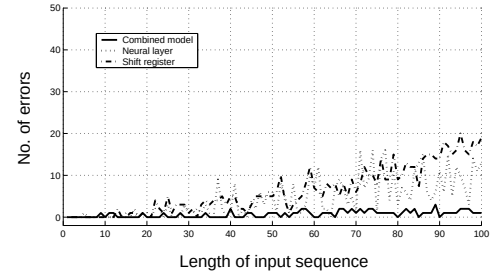


Fig. 8. Comparison of the performance of three types of sequence memories: Shift register (dashed and dotted line), neural layer(dotted) and the combined model(solid line). Optimal parameters have been used. The combined model performs better (least number of errors) than the others.

VII. CONCLUSION

We have developed a sequence machine out of asynchronous spiking neurons, that is scalable and error tolerant. We have discussed some of the problems in such modelling and our solutions.

REFERENCES

- [1] S.B. Furber, J.M. Cumpstey, W.J. Bainbridge and S. Temple. Sparse distributed memory using N-of-M codes. *Neural Networks* (10), 2004.
- [2] W. Maass and C.M. Bishop (eds.) *Pulsed Neural Networks* MIT Press, 1998
- [3] S. Thorpe, A. Delorme, R. Van Rullen. Spike based strategies for rapid processing. *Neural Networks* (14), 2001.
- [4] Pentti Kanerva. *Sparse Distributed Memory*. MIT Press, 1988.
- [5] T. Kohonen. Correlation matrix memories. *IEEE Transactions on Computers* (21), 1972.